

UNIVERSIDAD COMPLUTENSE DE MADRID

FACULTAD DE ESTUDIOS ESTADÍSTICOS

Máster en Big Data, Data Science & Inteligencia Artificial



TRABAJO DE FIN DE MÁSTER

**Asistente inteligente para análisis
aeronáutico (AEROGPT)**

DAVID CARRERA PINTOR

Curso académico 2024-2025

Contenido

1.	Introducción	3
2.	Marco Teórico	3
3.	Preparación de datos para sistemas RAG	4
3.1.	Fuentes de datos textuales	4
3.1.1.	Documentación técnica y normativa (PDFs).....	5
3.1.2.	Bases de datos de reportes y eventos (Excels).....	5
3.2.	Extracción y limpieza del texto.....	5
3.3.	Segmentación del texto (chunking)	6
3.4.	Generación de embeddings y representación vectorial	6
3.5.	Consideraciones metodológicas	6
4.	Preparación de datos y modelado para la predicción de la Vida Útil Restante (RUL)	7
4.1.	Conjunto de datos CMAPSS y escenario experimental	7
4.2.	Análisis inicial, limpieza e ingeniería de variables	8
4.3.	Normalización y construcción de secuencias temporales	8
4.4.	Evaluación de arquitecturas y validación cruzada.....	8
4.5.	Refinamiento y mejora del modelo GRU seleccionado	10
4.6.	Evaluación final y calibración post-predicción.....	10
4.7.	Conclusiones del capítulo.....	11
5.	Diseño de agentes AeroGPT	11
5.1.	Arquitectura común de los agentes basados en RAG	11
5.1.1.	Pipeline RAG compartido	11
5.1.2.	Representación vectorial, recuperación y modelos de lenguaje	11
5.1.3.	Gestión del estado y escalado entre agentes.....	12
5.2.	Agentes basados en RAG	12
5.3.	Agentes orientados a predicción de Vida Útil Restante (RUL).....	13
5.3.1.	Agente de Preprocesado RUL (PreRulAgent)	13
5.3.2.	Agente de Predicción RUL (RulAgent).....	14
5.4.	Agentes generales y de control	14
5.5.	Conclusiones del capítulo.....	15
6.	Orquestación multiagente y ejecución del sistema	15
6.1.	Arquitectura de orquestación	15
6.2.	Mecanismo de routing dinámico	16
6.3.	Gestión operativa del estado y control del flujo de ejecución	16
6.3.1.	Estrategias de limpieza del estado	17

6.4.	Encadenamiento condicional entre agentes	17
6.5.	Composición final de resultados	17
6.6.	Ciclo de ejecución del sistema	17
6.7.	Ventajas del enfoque multiagente	18
7.	Resultados y experimentación	18
7.1.	Definición del escenario experimental	18
7.2.	Flujo de ejecución y decisiones internas	19
7.2.1.	Preprocesado de datos (PreRulAgent).....	19
7.2.2.	Predicción de RUL (RulAgent)	19
7.3.	Evaluación de criticidad y escalado automático.....	19
7.4.	Recomendaciones de reparación.....	20
7.5.	Síntesis final y visualización	20
7.6.	Discusión	20
8.	Conclusiones generales y líneas futuras.....	20
9.	Bibliografía	22
10.	Anexos	23

1. Introducción

En los últimos años, la Ciencia de Datos y la Inteligencia Artificial han cobrado una relevancia creciente en sistemas industriales complejos, donde conviven grandes volúmenes de datos estructurados (por ejemplo, series temporales multivariantes de sensores) y extensas colecciones de documentación técnica y normativa. La integración de datos estructurados y documentación técnica es clave para mejorar la toma de decisiones en mantenimiento predictivo, evaluación de riesgo y cumplimiento normativo.

Este Trabajo Fin de Máster propone el diseño e implementación de **AeroGPT**, un sistema modular orientado a la **asistencia inteligente en consultas técnicas, predictivas y operacionales**, que integra modelos de predicción de vida útil restante (*Remaining Useful Life*, RUL) sobre series temporales de sensores con mecanismos de recuperación y razonamiento documental basados en *Retrieval-Augmented Generation* (RAG). La contribución central es una arquitectura **multiagente** que encapsula pipelines predictivos y documentales, coordinándolos de forma condicional y trazable.

Los objetivos específicos son:

1. Evaluar modelos de regresión y aprendizaje profundo para la estimación de la vida útil restante (RUL).
2. Diseñar un pipeline de ingestión, indexado y recuperación documental basado en RAG.
3. Encapsular los distintos pipelines analíticos en agentes especializados orientados a datos.
4. Integrar y evaluar el funcionamiento de un sistema multiagente con orquestación dinámica.

Como caso de estudio se emplea el dominio aeronáutico (p. ej., dataset CMAPSS), aunque las metodologías propuestas son aplicables a otros sectores industriales.

2. Marco Teórico

AeroGPT se fundamenta en tres pilares tecnológicos principales: **(1) modelos de lenguaje de gran escala para el razonamiento técnico-documental**, **(2) aprendizaje profundo aplicado a series temporales para la predicción de la Vida Útil Restante (Remaining Useful Life, RUL)** y **(3) arquitecturas multiagente para la coordinación de procesos analíticos heterogéneos**. La integración de estos enfoques permite abordar de forma unificada problemas complejos en entornos industriales regulados, donde conviven datos estructurados y grandes volúmenes de documentación técnica y normativa.

En el núcleo del sistema se encuentran los **modelos de lenguaje de gran escala (LLMs)**, integrados mediante el framework **LangChain**, que proporciona una abstracción robusta para gestionar prompts, cadenas de razonamiento y llamadas a modelos externos.

La coordinación de los agentes inteligentes se implementa mediante **LangGraph**, que modela la ejecución del sistema como un grafo de estados con transiciones condicionales. Cada agente actúa de forma autónoma, compartiendo un estado común. Los grafos de estados permiten flujos de ejecución no lineales y adaptativos, adecuados para escenarios técnicos complejos donde el análisis se escala según la severidad o el contexto operacional.

Una parte esencial del sistema es la explotación de **conocimiento documental** mediante **Retrieval-Augmented Generation (RAG)**. Este paradigma permite complementar el conocimiento paramétrico de los LLMs con información externa recuperada dinámicamente.

Mediante la generación de **embeddings vectoriales** y su almacenamiento en bases de datos especializadas como **FAISS**, se realiza búsqueda semántica eficiente sobre grandes volúmenes documentales, reduciendo el riesgo de alucinaciones y mejorando la fundamentación de las respuestas generadas.

El pipeline RAG incluye la extracción, limpieza y segmentación de documentos, así como la recuperación de fragmentos relevantes mediante estrategias de similitud semántica y **Maximum Marginal Relevance (MMR)**. Estas técnicas permiten equilibrar relevancia y diversidad informativa, aspecto crítico en dominios como la seguridad aeronáutica, donde es necesario considerar múltiples fuentes y perspectivas técnicas.

En paralelo, AeroGPT incorpora un núcleo analítico basado en aprendizaje profundo para la predicción de la **Vida Útil Restante (RUL)** de motores aeronáuticos a partir de series temporales multivariantes de sensores. Este problema constituye un caso de uso representativo del mantenimiento predictivo, donde la capacidad de anticipar fallos permite optimizar la planificación de inspecciones y reducir riesgos operacionales.

Para este fin se emplean redes neuronales recurrentes del **tipo Gated Recurrent Unit (GRU)**, adecuadas para modelar dependencias temporales y patrones de degradación progresiva. Estas arquitecturas presentan un compromiso favorable entre capacidad de modelado y estabilidad frente al sobreajuste, especialmente en escenarios con variabilidad operativa. El modelo GRU se implementa en **PyTorch** y se entrena utilizando técnicas estándar de regularización, incluyendo **early stopping**, ajuste dinámico del learning rate mediante **ReduceLROnPlateau** y regularización L2 mediante **weight decay**.

Dado que pequeñas desviaciones en la predicción de RUL pueden tener consecuencias operacionales significativas, se incorpora una **calibración post-predicción** mediante regresión lineal. Esta técnica permite corregir sesgos sistemáticos en la escala de las predicciones sin modificar la capacidad predictiva del modelo ni introducir dependencia con los datos de test, práctica habitual en contextos industriales de alta criticidad.

Para la interacción con el usuario y la visualización de resultados se implementa una interfaz de producción desarrollada con **Streamlit**, que sirve como punto de acceso principal para los usuarios finales del sistema. Esta interfaz web permite la consulta interactiva y visualización de resultados sin afectar a la lógica interna de los agentes ni al proceso de toma de decisiones.

En conjunto, el marco teórico de AeroGPT integra técnicas de procesamiento del lenguaje natural, aprendizaje profundo y sistemas multiagente para dar soporte a un dominio crítico como el aeronáutico, donde la robustez, la trazabilidad y la interpretabilidad constituyen requisitos fundamentales para la adopción de soluciones basadas en Inteligencia Artificial.

3. Preparación de datos para sistemas RAG

Este capítulo describe el proceso de preparación de los datos textuales utilizados por los **agentes basados en Retrieval-Augmented Generation (RAG)**. Dicho proceso constituye un conjunto de pipelines orientados a **transformar información no estructurada y semiestructurada en representaciones vectoriales** que permitan su recuperación semántica eficiente mediante técnicas de búsqueda por similitud.

La eficacia de un sistema RAG depende de forma directa de la calidad del preprocesado: la selección adecuada de las fuentes, la limpieza del texto, la segmentación coherente de los documentos y la **generación de embeddings representativos**. A continuación, se detalla de manera metodológica cómo se ha llevado a cabo este proceso en el presente trabajo.

Es importante señalar que este capítulo se centra exclusivamente en datos textuales, mientras que el tratamiento de datos numéricos y series temporales para la predicción de RUL se aborda de forma independiente en el Capítulo 4, manteniendo así una separación clara entre ambos tipos de información.

3.1. Fuentes de datos textuales

Los agentes basados en RAG utilizan datos heterogéneos con distintos niveles de estructuración y formalidad, procedentes de dos grandes categorías: documentación técnica y normativa (formato PDF), y

bases de datos de reportes operacionales (Excels). La *Tabla A.1* resume las fuentes empleadas, su formato y el vectorstore asociado.

3.1.1. Documentación técnica y normativa (PDFs)

Esta categoría está compuesta por documentos técnicos y regulatorios emitidos por organismos oficiales y fabricantes. Se trata de textos mayoritariamente no estructurados, con lenguaje técnico especializado y presencia frecuente de encabezados, numeraciones seccionales, referencias cruzadas, tablas y figuras.

Las principales fuentes empleadas son:

- FAA Advisory Circulars (ACs), que contienen guías técnicas, criterios de reparación y procedimientos de mantenimiento organizados por partes regulatorias.
- Reglamentos y especificaciones de certificación de EASA, incluyendo CS-23, CS-25, Part-145, Part-M y documentos AMC/GM, con un alto componente jurídico-técnico.
- Airbus FAST Magazine, que aporta conocimiento operativo mediante estudios de caso y recomendaciones basadas en experiencia real.

Estos documentos constituyen la base de conocimiento formal para consultas técnicas y regulatorias. Los archivos PDF se procesan mediante un pipeline específico (Pipeline A) y se almacenan en dos vectorstores diferenciados: **regulatory_store** y **technical_store**.

3.1.2. Bases de datos de reportes y eventos (Excels)

La segunda categoría está formada por bases de datos operacionales que combinan metadatos estructurados con narrativas textuales introducidas por operadores humanos. Estas fuentes aportan conocimiento basado en experiencia operativa real.

En particular, se emplearon:

- NASA Aviation Safety Reporting System (ASRS), con reportes de seguridad operacional que incluyen metadatos y descripciones narrativas de incidentes.
- FAA Service Difficulty Reports (SDR), que describen fallos mecánicos, componentes afectados y acciones correctivas.

Estos datos se procesan mediante un pipeline independiente (Pipeline B) y se almacenan en los vectorstores **asrs_store** y **sdr_store**, respectivamente.

3.2. Extracción y limpieza del texto

El proceso de extracción y limpieza varía de forma significativa en función del tipo de fuente, por lo que se implementan dos pipelines diferenciados.

- **Documentos PDF (Pipeline A):** se utiliza la biblioteca `pdfplumber` para extraer texto sin alterar la estructura original. Este enfoque preserva encabezados y numeraciones, lo que resulta fundamental para mantener coherencia semántica durante la segmentación posterior.
- **Bases de datos Excel (Pipeline B):** el procesamiento de datos tabulares requiere operaciones adicionales de limpieza y estructuración. En el caso de ASRS, los encabezados multinivel son aplanados para obtener una estructura coherente. Posteriormente se eliminan etiquetas HTML, se normalizan espacios en blanco y se seleccionan únicamente las columnas relevantes mediante diccionarios de mapeo.

Cada fila se transforma en un documento textual estructurado que combina múltiples campos con etiquetas explícitas, por ejemplo:

```
{
  Aircraft: AIRBUS 400M,
  FAR Part: 121,
  Flight Phase: "Descent",
  System / Component: "Hydraulic System",
  Primary Problem: "Fluid Leak",
  Anomaly: "Aircraft Equipment Problem",
  NARRATIVE: [texto narrativo],
  SYNOPSIS: [resumen]
}
```

En el caso de SDR se aplica un proceso análogo, adaptado a la estructura específica de sus campos. A diferencia del pipeline de PDFs, los documentos procedentes de Excel no requieren segmentación adicional, ya que cada registro constituye una unidad semántica completa.

3.3. Segmentación del texto (chunking)

La segmentación se aplica únicamente a documentos PDF debido a las limitaciones de longitud de los modelos de embeddings. Los textos se dividen en fragmentos controlados para preservar el contexto semántico y evitar la ruptura de unidades de significado.

Parámetros empleados:

- Tamaño de chunk: 1200 caracteres
- Solapamiento: 150 caracteres

Justificación experimental:

- Un tamaño de 1200 caracteres permite cubrir párrafos completos con sentido coherente. Valores menores (500–800) generaban fragmentos excesivamente cortos, mientras que valores mayores (2000) reducían la granularidad y aumentaban el coste computacional.
- Un solapamiento de 150 caracteres asegura que frases situadas en los límites de fragmento no pierdan contexto. Valores inferiores afectaban a la coherencia y valores superiores incrementaban la redundancia sin mejoras apreciables.

Los fragmentos resultantes se almacenan junto con su metadata en archivos serializados para su posterior uso en los vectorstores mediante archivos pickle.

3.4. Generación de embeddings y representación vectorial

Tanto los fragmentos de PDF como los documentos estructurados procedentes de Excel se transforman en representaciones vectoriales mediante el modelo **text-embedding-ada-002** de OpenAI, integrado a través del framework **LangChain**.

Las principales características de esta etapa son:

- **Vectorstore utilizado:** FAISS Flat L2 para búsqueda exacta por similitud.
- **Persistencia local:** índices almacenados en archivos `index.faiss` e `index.pkl`.
- **Gestión de metadata:** cada documento se almacena como un objeto `Document` con información contextual relevante.

La metadata desempeña un papel clave para filtrar y contextualizar resultados, incluyendo campos como sistema afectado, fase de vuelo, tipo de problema o referencia normativa.

3.5. Consideraciones metodológicas

Para la recuperación de información se emplea la estrategia **Maximum Marginal Relevance (MMR)**, que permite equilibrar la relevancia de los documentos recuperados con la diversidad del contexto

proporcionado al modelo. Esta técnica resulta especialmente útil en dominios técnicos donde una misma consulta puede estar relacionada con múltiples fuentes complementarias.

La configuración empleada en el sistema es la siguiente:

- Número final de documentos recuperados: **k = 5**
- Número inicial de candidatos: **fetch_k = 20**
- Parámetro de equilibrio relevancia–diversidad: **$\lambda = 0.7$**

Estos valores fueron seleccionados tras pruebas empíricas preliminares orientadas a maximizar la pertinencia de los resultados sin introducir redundancias excesivas. Configuraciones con valores menores de λ tendían a priorizar excesivamente la diversidad, mientras que valores superiores reducían la variedad de información recuperada.

Además de la recuperación semántica, se aplican **filtros basados en metadatos** cuando están disponibles, tales como sistema afectado, fase de vuelo o parte regulatoria aplicable. Esta combinación de búsqueda vectorial y filtrado estructurado incrementa la precisión de las respuestas y reduce el ruido documental, especialmente en consultas técnicas de carácter específico.

En conjunto, estas decisiones metodológicas permiten que los agentes basados en RAG dispongan de un contexto informativo relevante, diverso y trazable, asegurando que las respuestas generadas estén fundamentadas en evidencia documental adecuada.

4. Preparación de datos y modelado para la predicción de la Vida Útil Restante (RUL)

Además de la explotación de información textual mediante técnicas de RAG, este trabajo aborda un problema central de la analítica predictiva basada en datos: la estimación de la Vida Útil Restante (Remaining Useful Life, RUL) de motores aeronáuticos a partir de datos multivariantes de sensores. Este problema constituye un caso de uso representativo de técnicas avanzadas de ciencia de datos, incluyendo limpieza de información, ingeniería de variables, modelado predictivo y evaluación comparativa de arquitecturas.

En este capítulo se describe el pipeline completo aplicado al conjunto de datos CMAPSS, desde la preparación inicial de los datos hasta la selección, ajuste y evaluación final del modelo predictivo. El objetivo no es únicamente maximizar el rendimiento puntual, sino obtener un modelo estable y generalizable, capaz de integrarse de forma fiable en el sistema multiagente.

4.1. Conjunto de datos CMAPSS y escenario experimental

El conjunto de datos CMAPSS, desarrollado por la NASA, simula la degradación progresiva de motores aeronáuticos bajo distintos regímenes operativos y modos de fallo. Se trata de un dataset sintético ampliamente utilizado como referencia en problemas de predicción de RUL.

El dataset se organiza en cuatro subconjuntos independientes (FD001–FD004), que difieren en complejidad según el número de condiciones operativas y los modos de fallo considerados. Cada observación corresponde a un ciclo de operación e incluye un identificador de unidad, el número de ciclo, tres variables de configuración y veintiuna señales de sensores.

Los subconjuntos se caracterizan de la siguiente manera:

- **FD001:** 100 trayectorias de entrenamiento y 100 de test, **una condición operativa y un único modo de fallo (HPC)**. Escenario más sencillo.

- **FD002:** 260 entrenamiento y 259 test, **seis condiciones operativas y un modo de fallo** (HPC).
- **FD003:** 100 entrenamiento y 100 test, **una condición operativa y dos modos de fallo** (HPC y Fan).
- **FD004:** 248 entrenamiento y 249 test, **seis condiciones operativas y dos modos de fallo** (HPC y Fan). Escenario más complejo.

Esta diferenciación resulta clave para la interpretación de los resultados, ya que el **incremento de variabilidad operacional y la coexistencia de múltiples modos de fallo** afectan directamente a la **estabilidad y capacidad de generalización** de los modelos predictivos.

4.2. Análisis inicial, limpieza e ingeniería de variables

Como paso previo al modelado se realizó un análisis sistemático de la calidad de los datos con el objetivo de garantizar la coherencia del pipeline. Este análisis incluyó la verificación de valores nulos, la comprobación de la correspondencia entre unidades de test y valores reales de RUL y la identificación de variables constantes o con varianza nula.

Como resultado de este análisis se detectó que determinadas señales de sensores no aportaban información discriminativa en función del subconjunto FD, por lo que fueron eliminadas automáticamente. Esta limpieza inicial permitió reducir ruido y mejorar la estabilidad de las predicciones.

No se aplicaron técnicas explícitas de reducción de dimensionalidad como PCA, dado que los modelos recurrentes empleados aprenden representaciones latentes directamente a partir de las secuencias temporales multivariantes.

4.3. Normalización y construcción de secuencias temporales

Las señales de sensores presentan escalas heterogéneas, por lo que se aplicó un proceso de normalización mediante *StandardScaler*. El escalador se ajustó exclusivamente con los datos de entrenamiento y se aplicó posteriormente al conjunto de test, evitando cualquier forma de fuga de información entre conjuntos.

Para adaptar los datos a modelos recurrentes, las observaciones individuales se transformaron en secuencias temporales de longitud fija mediante una **ventana deslizante**. En el conjunto de entrenamiento se generan múltiples secuencias por unidad, mientras que en el conjunto de test se utiliza exclusivamente la última ventana disponible, reflejando un escenario real de aplicación.

Cuando una unidad dispone de menos de 30 ciclos, se aplica un mecanismo de **generación sintética de historial** que incorpora ruido gaussiano controlado y suavizado progresivo, con el objetivo de completar la ventana temporal sin introducir patrones artificiales abruptos.

4.4. Evaluación de arquitecturas y validación cruzada

Durante la fase exploratoria se evaluaron tres arquitecturas de redes neuronales recurrentes: **RNN, LSTM y GRU**. Para analizar su estabilidad se empleó un esquema de **validación cruzada por grupos (GroupKFold)**, garantizando que todas las secuencias pertenecientes a una misma unidad de motor quedaran siempre en el mismo pliegue.

Para cada combinación subconjunto FD–arquitectura se calcularon métricas agregadas de **RMSE, MAE y NASA score**, obteniendo tanto el valor medio como la desviación estándar a lo largo de los pliegues. Esta última es la métrica de referencia para el dataset CMAPSS, ya que penaliza de forma asimétrica las predicciones tardías, consideradas más críticas en un contexto operacional.

La *Ilustración 1* muestra la distribución del RMSE por arquitectura evidenciando patrones consistentes en todos los subconjuntos FD.

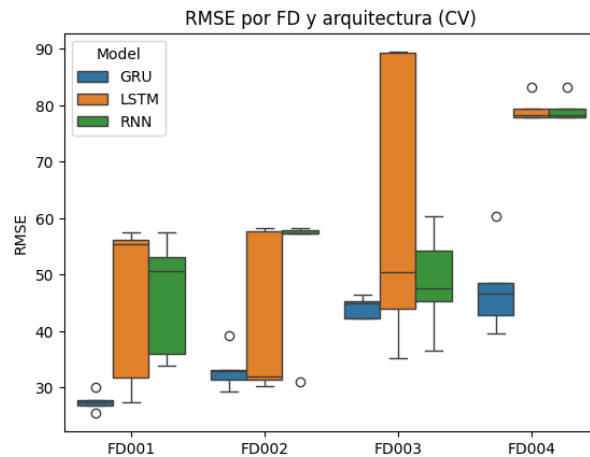


Ilustración 1: Distribución del RMSE por arquitectura y subconjunto FD en validación cruzada

Las **RNN clásicas** presentan valores medios de error elevados junto con desviaciones estándar significativas, evidenciando una alta sensibilidad a la partición de los datos. Las **LSTM** alcanzan en algunos casos errores medios competitivos, pero acompañados de una dispersión elevada, especialmente en los escenarios más complejos (FD003 y FD004). Por el contrario, las **GRU** muestran de forma sistemática los **menores valores medios de RMSE** junto con una **menor variabilidad entre pliegues**, indicando un comportamiento más estable y robusto.

La decisión final de arquitectura se basó en un ranking agregado construido a partir de las métricas de validación cruzada, normalizando RMSE y NASA score y combinándolos en un score ponderado (**0.6 RMSE, 0.4 NASA score**). Este criterio prioriza un equilibrio entre precisión global y riesgo operativo:

FD	Model	RMSE_mean	RMSE_std	MAE_mean	MAE_std	NASA_mean	NASA_std	n_folds	RMSE_mean_norm	NASA_mean_norm	Score	Rank
FD001	GRU	27.562785	1.682126	17.525702	0.402334	7.371049e+02	9.751516e+02	5	0.000000	0.000000	0.000000	1
FD001	RNN	46.242056	10.657086	35.698716	10.573949	6.380441e+03	9.449553e+03	5	1.000000	0.785053	0.914021	2
FD001	LSTM	45.623546	14.722852	34.912282	15.187553	7.925579e+03	8.679707e+03	5	0.966888	1.000000	0.980133	3
FD002	GRU	33.184749	3.700420	23.206709	3.757819	1.708239e+03	1.353807e+03	5	0.000000	0.000000	0.000000	1
FD002	LSTM	41.920875	14.661640	31.618343	13.830051	4.782188e+04	9.132342e+04	5	0.455605	0.709742	0.557260	2
FD002	RNN	52.359519	11.910989	41.371909	11.790880	6.668061e+04	8.649289e+04	5	1.000000	1.000000	1.000000	3
FD003	GRU	44.261758	1.938775	27.959302	1.746911	2.724284e+04	2.536591e+04	5	0.000000	0.000000	0.000000	1
FD003	RNN	48.848965	9.049530	33.155228	7.764761	1.456055e+05	2.735693e+05	5	0.262957	0.001783	0.158488	2
FD003	LSTM	61.706452	25.854302	44.501255	23.153524	6.640884e+07	9.994731e+07	5	1.000000	1.000000	1.000000	3
FD004	GRU	47.602444	7.921674	33.177975	6.442484	5.450606e+06	1.198237e+07	5	0.000000	0.000000	0.000000	1
FD004	LSTM	79.373429	2.216603	63.585716	0.820457	1.456672e+09	3.182727e+09	5	1.000000	0.958949	0.983580	2
FD004	RNN	79.373327	2.218989	63.562652	0.793738	1.518796e+09	3.321340e+09	5	0.999997	1.000000	0.999998	3

Tabla 1: Resumen de métricas agregadas de validación cruzada y test por arquitectura y subconjunto FD

Los valores de la *Tabla 1* evidencian de forma consistente que:

- En los subconjuntos **FD001 y FD002**, la arquitectura **GRU** obtiene los menores valores de **score** agregado, acompañados de una baja variabilidad entre pliegues, ocupando sistemáticamente la primera posición en el ranking.
- En **FD003 y FD004**, caracterizados por una mayor complejidad operacional y múltiples modos de fallo, las **GRU** mantienen un **comportamiento estable**, mientras que las arquitecturas **RNN y LSTM** presentan **incrementos significativos** tanto en el error medio como en la variabilidad, especialmente en el **NASA score**.
- Aunque RNN y LSTM alcanzan valores competitivos en el conjunto de test, su elevada inestabilidad en validación cruzada limita su fiabilidad en escenarios operativos reales.

De forma agregada, el análisis demuestra que las **GRU** ofrecen el mejor equilibrio entre **precisión, estabilidad y robustez**, manteniendo un rendimiento consistente en todos los subconjuntos FD.

Por estos motivos, esta arquitectura se seleccionó como modelo final para el sistema de predicción de *Remaining Useful Life* (RUL) desarrollado, debido a su capacidad para **generalizar** y su **fiabilidad operativa**

En la *Ilustración A.1*, se puede observar la comparación entre los valores reales y predichos de RUL para cada una de las arquitecturas y FD.

4.5. Refinamiento y mejora del modelo GRU seleccionado

Tras seleccionar la arquitectura GRU, se llevó a cabo una fase de refinamiento orientada a maximizar su rendimiento y estabilidad. La arquitectura definitiva empleada es la siguiente:

- **Modelo:** GRU profundo con 4 capas recurrentes, organizadas en dos bloques GRU apilados:
 - **GRU 1:** input_dim=24, hidden_dim=256, num_layers=2, dropout=0.3, batch_first=true.
 - **GRU 2:** input_dim=256, hidden_dim=128, num_layers=2, dropout=0.3, batch_first=true.
 - **Capa lineal de salida (fully connected):** 128 → 1 (predicción de RUL)
- **Ventana temporal de 30 timesteps y 24 variables de entrada.**

El entrenamiento se realizó con función de pérdida MSE, optimizador Adam (learning rate 0.001), regularización L2, ReduceLROnPlateau y early stopping con paciencia de 30 épocas. Se utilizó una partición interna 80/20 para validación, manteniendo el conjunto de test completamente independiente.

Este refinamiento permitió obtener un modelo estable y capaz de capturar la dinámica temporal de los sensores. No obstante, se observaron sesgos sistemáticos en escenarios complejos, lo que motivó la implementación de calibración post-predicción.

4.6. Evaluación final y calibración post-predicción

El modelo GRU refinado se evaluó sobre los conjuntos de test utilizando exclusivamente la **última ventana temporal de cada unidad**. Para corregir los sesgos observados, especialmente en FD003 y FD004, se aplicó una **calibración lineal post-predicción** mediante regresión lineal (*LinearRegression de scikit-learn*).

Puntos clave sobre la calibración post-predicción:

- Se aplica únicamente sobre las salidas del modelo ya entrenado.
- No modifica los pesos del GRU ni introduce capacidad predictiva adicional.
- Evita cualquier forma de data leakage.
- Se emplean calibradores específicos por FD y un calibrador global de respaldo.

FD	MAE	MAE Post-Calib	RMSE	RMSE Post-Calib	NASA score	NASA score Post-Calib
FD001	19.664	14.541	25.796	18.792	3,387.041	553.052
FD002	24.216	24.098	33.420	32.219	110,167.384	53,437.917
FD003	35.854	19.340	55.232	24.313	124,225,466.781	3,977.614
FD004	32.097	28.242	42.350	34.987	1,060,574.025	19,813.494

Tabla 2. Comparativa de métricas pre y post calibración por FD.

Los resultados (Tabla 2) mostraron reducciones significativas del MAE, RMSE y NASA score en todos los subconjuntos, especialmente en FD003 y FD004, confirmando la utilidad de este ajuste para escenarios operativos reales.

En los anexos se incluyen gráficas de **RUL real vs predicho** para cada FD, mostrando la mejora tras la calibración (Ilustración A.2) y una tabla resumen comparativa con de las métricas pre y post calibración (Tabla A.2).

4.7. Conclusiones del capítulo

El pipeline desarrollado combina limpieza rigurosa de datos, ingeniería de variables orientada a series temporales y validación comparativa de arquitecturas. La selección de un modelo GRU estable, junto con la calibración post-predicción, permitió obtener un sistema de predicción de RUL competitivo y robusto.

Los resultados demuestran que el modelo final es capaz de generalizar a distintas condiciones operativas y modos de fallo, proporcionando estimaciones fiables que pueden integrarse de forma segura en el sistema multiagente para soporte a la decisión en mantenimiento predictivo.

5. Diseño de agentes AeroGPT

Este capítulo describe el diseño y desarrollo de los agentes inteligentes que conforman el sistema AeroGPT, analizados **de forma individual e independiente**, sin abordar su integración u orquestación conjunta, que se tratará en el capítulo siguiente. Cada agente se concibe como un **componente autónomo y especializado** en un tipo concreto de datos y técnicas analíticas, siguiendo un enfoque **modular** alineado con arquitecturas modernas basadas en agentes.

Antes de detallar cada agente, es importante describir una serie de principios y mecanismos comunes que estructuran su funcionamiento.

- **Gestión de estado compartido:** todos los agentes operan sobre un objeto (*state*) común, que actúa como memoria estructurada del sistema.
- **Uso de prompts explícitos y controlados:** cada agente utiliza instrucciones explícitas que delimitan su rol, salida y ámbito de actuación, minimizando el riesgo de respuestas no fundamentadas.
- **Separación entre razonamiento y ejecución:** las decisiones se toman mediante LLMs, mientras que las operaciones críticas (cálculo, *parsing*, validación) se ejecutan mediante código determinista.
- **Trazabilidad y robustez:** se incorporan mecanismos de *logging*, manejo de excepciones y salidas conservadoras para cada agente.

5.1. Arquitectura común de los agentes basados en RAG

Los agentes **Técnico, Regulación, Criticidad y Reparación** comparten una **arquitectura y flujo operativo común**, basada en Retrieval-Augmented Generation (RAG), que garantiza trazabilidad y control del comportamiento del modelo.

5.1.1. Pipeline RAG compartido

Todos los agentes RAG siguen el mismo pipeline lógico:

1. **Recepción de la consulta del usuario** junto con el estado actual del sistema.
2. **Recuperación semántica** desde una o varias bases vectoriales FAISS.
3. **Construcción de un contexto** textual a partir de los fragmentos recuperados.
4. **Generación de la respuesta** mediante un modelo de lenguaje restringido al contexto.
5. **Producción de salida** en formato texto o *JSON* estructurado. La salida de cada agente se almacena de forma estructurada en el objeto *state*, permitiendo su recuperación por otros agentes sin pérdida de contexto.
6. **Evaluación de criticidad y decisión de escalado** a otro agente si procede.

5.1.2. Representación vectorial, recuperación y modelos de lenguaje

Todos los agentes RAG utilizan:

- **OpenAIEmbeddings** para la generación de *embeddings* semánticos (representaciones vectoriales).

- **FAISS** como base de datos vectorial para búsquedas eficientes por similitud.

Según el tipo de agente, se emplean dos estrategias de recuperación:

- **similarity_search**: prioriza máxima relevancia directa.
- **max_marginal_relevance_search (MMR)**: prioriza la diversidad informativa.
- Para la generación de respuestas se utilizan dos configuraciones del modelo de lenguaje **gpt-4.1-nano**:
 - **llm_deterministic** (temp = 0): clasificación, decisiones críticas y generación de JSON estructurado.
 - **llm_creative** (temp = 0.7): análisis contextualizado bajo restricciones estrictas de contexto

5.1.3. Gestión del estado y escalado entre agentes

Todos los agentes leen y escriben sobre un estado compartido (**state**), definido mediante la clase **AgentState** como modelo **Pydantic** que actúa como memoria central del sistema. Este diseño permite coordinar el flujo de ejecución sin acoplamiento directo entre agentes.

La *Tabla 3* resume los principales campos del estado, organizados por categorías funcionales.

Categoría	Atributos AgentState	Propósito
Mensajes	messages	Historial de conversación de LangChain (HumanMessage, AIMessage)
Control de Enrutamiento	decision, next_agent, needs_followup, source	Control del flujo y la transición entre agentes
Salidas de los Agentes	regulacion, criticidad, reparacion, rul, tecnico, general_notes, final_response	Resultados de análisis específicos de dominio generados por cada agente
Específico de RUL	pre_rul_data, modelo_seleccionado	Datos de sensores CMAPSS y modelo seleccionado (FD001–FD004)
Memoria e Interfaz de Usuario	conversation_summary, history_by_agent,	Memoria conversacional y analítica
Interfaz de Usuario	output_buffer, debug_buffer	Retroalimentación para la interfaz de usuario

Tabla 3: Campos de AgentState con las categorías y propósito correspondientes

Este diseño permite:

- Encadenamiento condicional entre agentes (*control de enrutamiento*).
- Auditoría completa del razonamiento.
- Separación estricta entre especialidades.
- Evolución explícita del estado entre decisiones sucesivas.

5.2. Agentes basados en RAG

Agente Técnico: responsable de análisis técnicos y explicativo sobre sistemas aeronáuticos y mantenimiento. Recupera información desde documentación técnica y genera respuestas orientadas a la comprensión operativa, evitando recomendaciones de acción directa (*salida en state.tecnico*).

Agente de Regulación: especializado en consultas sobre el cumplimiento normativo FAA/EASA (AD, CS, Part-M/145, ACs). Produce análisis estructurados en formato JSON que incluyen referencias regulatorias, limitaciones operativas y riesgos de cumplimiento (*salida en state.regulacion, ejemplo ilustrativo en Tabla A.3*).

Agente de Criticidad: evalúa la severidad y el riesgo operacional a partir de múltiples fuentes de información. Clasifica el impacto sobre la seguridad y determina si una aeronave es apta para despacho

produciendo un JSON con sistema afectado, fase de vuelo, severidad, riesgo y recomendaciones (*salida en state.criticidad, ejemplo ilustrativo en Tabla A.4*).

Agente de Reparación: genera planes de mantenimiento accionables para escenarios de mantenimiento aeronáutico al sintetizar la información disponible y consultar cuatro almacenes vectoriales especializados (ASRS, SDR, Regulatory, Technical). Produce recomendaciones de mantenimiento estructuradas que incluyen identificación del sistema, evaluación de severidad, acciones recomendadas y referencias técnicas en formato JSON (*salida en state.reparacion, ejemplo ilustrativo en Tabla A.5*).

En la siguiente tabla, se puede observar de manera esquemática los datos que consume cada agente, junto con las técnicas de recuperación que emplea:

Agente	Datos consumidos	Técnicas
Técnico	- Documentación AMM/ACs, guías EASA/FAA	FAISS + similarity_search, llm_creative + llm_deterministic
Regulación	- FAA/EASA (AD, CS, Part-M/145, Acs)	FAISS + MMR, llm_deterministic
Criticidad	- ASRS, SDR, documentación técnica y normativa. - Salidas agentes previos	MMR multi-vectorstores, llm_deterministic
Reparación	- ASRS, SDR, REGULATORY y TECHNICAL. - Salida agentes previos	MMR multi-vectorstores, llm_deterministic

Tabla 4: Tabla resumen características individuales de los agentes

5.3. Agentes orientados a predicción de Vida Útil Restante (RUL)

Los agentes orientados a RUL operan exclusivamente sobre **datos numéricos multivariantes y series temporales de sensores** (desarrollados en el *Capítulo 4*). Su objetivo es proporcionar estimaciones cuantitativas de la vida útil restante de activos, junto con una interpretación técnica del estado de degradación, integrable dentro del sistema multiagente.

A diferencia de los agentes RAG, estos agentes no emplean recuperación semántica ni embeddings, sino que se apoyan en **pipelines de ingeniería de datos, modelos de Deep Learning y validación predictiva**, constituyendo el núcleo analítico del sistema.

5.3.1. Agente de Preprocesado RUL (PreRulAgent)

Este agente es el punto de entrada para el flujo de trabajo de predicción de RUL. Su responsabilidad principal es orquestar la recopilación y acumulación de datos de sensores CMAPSS y modelo seleccionado a partir de descripciones en lenguaje natural antes de iniciar la predicción de RUL. El agente implementa una interfaz conversacional que interpreta la intención del usuario, gestiona un DataFrame de mediciones históricas de sensores y enruta al **RulAgent** cuando hay datos suficientes para la predicción y se solicite su cálculo.

Entradas

- Mensaje del usuario en lenguaje natural.
- Estado previo de datos acumulados (*state.pre_rul_data* y *state.modelo_seleccionado*).

Tools utilizadas

- **extract_cmapss**: convierte texto natural del usuario en un JSON estructurado:

```
{
  "unidad": 1,           # Identificador del motor
  "tiempo_ciclos": 1,    # Ciclo de operación
  "setting_1": 10,       # 3 Configuraciones de motor
  "setting_2": 20,
  "setting_3": 30,
  "s_1": 0.5,            # 21 sensores del motor
  ... s_21
}
```

- **tool_output_to_df**: transforma el JSON en DataFrame CMAPSS.

Decisión de acción: mediante su prompt, el LLM clasifica la intención del usuario en una de las acciones:

- **Update**: sigue el siguiente flujo:

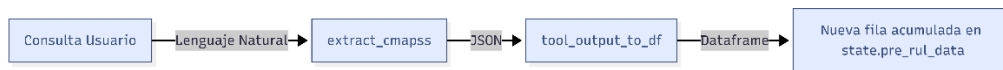


Ilustración 2: Flujo Update de PreRulAgent

- **Calculate:** valida si existen suficientes ciclos acumulados y deriva hacia el **RulAgent**.
- **Status:** devuelve información descriptiva del histórico almacenado.
- **Reset:** reinicia el DataFrame (nuevo motor o nueva unidad).
- **Chat:** responde a preguntas conceptuales sobre CMAPSS y RUL.

Manejo de errores: captura excepciones de extracción, parsing o concatenación y genera mensajes controlados.

Salida: estado actualizado en state.pre_rul_data y modelo seleccionado.

5.3.2. Agente de Predicción RUL (RulAgent)

Predice la Vida Útil Restante de motores aeronáuticos usando las redes neuronales GRU entrenadas. Produce explicaciones técnicas sobre la degradación del motor y enruta los casos críticos a la evaluación de seguridad.

Entradas

- state.pre_rul_data: DataFrame validado y preparado por el PreRulAgent.
- state.modelo_seleccionado: identificador CMAPSS (FD001–FD004).
- Modelos y artefactos en MODEL_DIR: best_model_{FD}.pth, scaler_{FD}.pkl, calib_{FD}.pkl, calib_global.pkl.

Métodos y técnicas aplicadas (pipeline interno)

- **Entrada mínima:** secuencia de 30 timesteps. Si es menor, se lleva a cabo una generación de filas sintéticas con ruido $\sigma=0.015$ y suavizado para completar la ventana.
- **Preprocesado:** imputación, escalado con StandardScaler específico (scaler_{FD}).
- **Ventaneo/reshape** → tensor (1, seq_len=30, features=24).
- **Inferencia:** Se carga el modelo GRU definido en el Capítulo 4.
- **Calibración:** con calib_{FD}.pkl o calib_global.
- **Validaciones:** comprobaciones de input, predicción no nula y rango lógico.

Salida

- **state.rul** = {"predicted_RUL": float, "text": str}
 - **predicted_RUL:** valor calibrado
 - **text:** explicación técnica generada por llm_creative integrando RUL numérico, sensores degradados y modos de fallo probables.

Degradación y clasificación

- ≥ 80 : Desgaste bajo.
- 40: Desgaste moderado (planificar inspección).
- 20: Desgaste significativo (evaluar inspección avanzada).
- 5: Riesgo elevado (monitorizar continuamente).
- ≤ 5 : ALERTA CRÍTICA (retirada inmediata recomendada).

Enrutamiento automático a CriticidadAgent si RUL < 20.

5.4. Agentes generales y de control

Los agentes General, Supervisor y Final cumplen funciones de interacción, enrutamiento y síntesis, garantizando coherencia global y correcta interacción con el usuario. Estos agentes no realizan ingeniería de datos ni modelado:

SupervisorAgent: Este agente introduce una decisión determinista respecto al enrutamiento inicial de la consulta del usuario al agente más pertinente mediante RAG.

GeneralAgent: respuestas informativas sin cálculo ni escalado. Este agente posee conocimientos sobre el sistema lo que le permite ofrecer resúmenes de la conversación o explicaciones sobre las funcionalidades de la aplicación, asistiendo al usuario en la comprensión de lo que se puede llevar a cabo con ella.

FinalAgent: Se encarga de realizar una síntesis final basada exclusivamente en las salidas generadas por los agentes anteriores. Este agente no añade nueva información, sino que compila lo recogido en interacciones previas para ofrecer un análisis completo, coherente y conciso.

5.5. Conclusiones del capítulo

En este capítulo se ha descrito el diseño de los agentes que conforman el sistema AeroGPT desde una perspectiva de Ciencia de Datos. Cada agente aplica técnicas específicas sobre tipos de datos bien definidos, produciendo salidas especializadas y coherentes con su función.

Los agentes basados en RAG operacionalizan conocimiento técnico y normativo mediante recuperación semántica controlada, garantizando respuestas fundamentadas y auditables, mientras que los agentes de predicción RUL constituyen el núcleo analítico del sistema, integrando ingeniería de datos y modelos de Deep Learning.

Finalmente, los agentes generales y de control proporcionan coherencia y gobernanza al flujo de interacción, asegurando que las salidas especializadas se sinteticen de forma consistente con la intención del usuario. La descomposición modular adoptada sienta las bases para la orquestación multiagente descrita en el capítulo siguiente.

6. Orquestación multiagente y ejecución del sistema

Este capítulo describe el mecanismo de orquestación multiagente que **permite integrar los agentes definidos en el Capítulo 5** dentro de un sistema operativo coherente, dinámico y trazable. A diferencia del capítulo anterior, aquí se aborda cómo los agentes se coordinan, encadenan y ejecutan mediante un grafo de estados, desde la recepción de la consulta del usuario hasta la generación de la respuesta final.

La orquestación constituye el elemento integrador que permite combinar analítica predictiva, recuperación semántica y reglas operacionales dentro de un único flujo de ejecución adaptativo.

6.1. Arquitectura de orquestación

El sistema AeroGPT implementa una arquitectura multiagente basada en **LangGraph**, donde la ejecución se modela como un **grafo de estados con transiciones condicionales**. Este enfoque permite definir flujos no lineales y adaptativos, alineados con la naturaleza progresiva de las consultas técnicas, operacionales y predictivas.

El núcleo de la arquitectura se materializa en la clase **GraphBuilder**, responsable de:

- Definir la topología del grafo.
- Registrar los nodos correspondientes a cada agente.
- Establecer las reglas de transición condicional entre ellos.

El grafo está compuesto por nueve nodos especializados, cada uno encapsulando una responsabilidad analítica concreta. Esta estructura desacopla completamente la lógica interna de cada agente, facilitando la extensibilidad, la depuración y la trazabilidad del sistema.

La *Ilustración 3* presenta el esquema completo de la orquestación del sistema, mostrando el recorrido de una consulta desde el **SupervisorAgent** hasta el **FinalAgent**, pasando por los agentes especializados según las decisiones tomadas durante la ejecución.

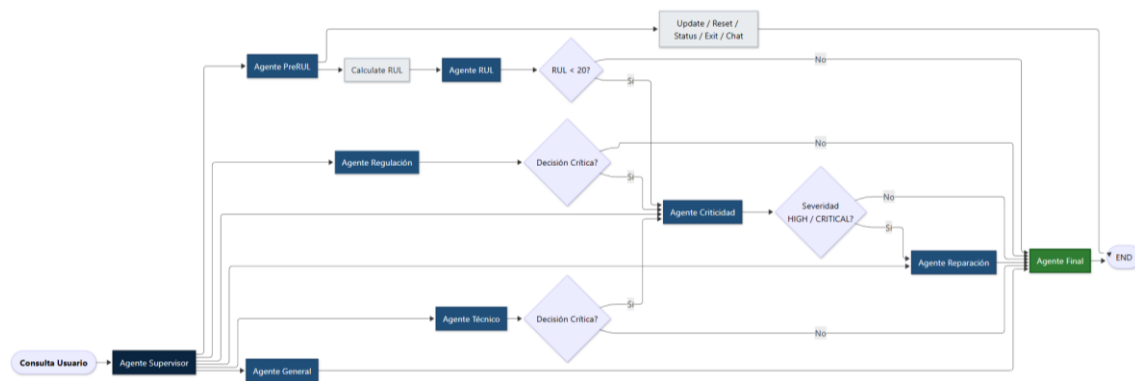


Ilustración 3: Grafo Arquitectura orquestación por agentes

6.2. Mecanismo de routing dinámico

La ejecución del grafo está gobernada por un mecanismo de routing dinámico, basado en decisiones condicionales explícitas. Estas decisiones permiten adaptar el flujo de análisis a la intención del usuario y a la severidad del caso evaluado.

Se distinguen dos tipos principales de decisiones:

1. **Decisión inicial del SupervisorAgent:** clasifica semánticamente la consulta del usuario y selecciona el agente especializado más adecuado como punto de entrada.
2. **Decisiones de seguimiento (follow-up):** permiten el encadenamiento automático de agentes cuando un análisis parcial detecta la necesidad de una evaluación adicional.

Este enfoque evita flujos rígidos y permite que el sistema incremente progresivamente la profundidad del análisis solo cuando resulta necesario.

6.3. Gestión operativa del estado y control del flujo de ejecución

Todos los agentes de AeroGPT operan sobre un estado compartido estructurado (state), que actúa como memoria común del sistema. En este apartado no se redefine dicha estructura, sino que se describe su uso operativo durante la ejecución del grafo multiagente.

En tiempo de ejecución, el estado cumple roles fundamentales:

1. Almacenar los resultados parciales generados por los agentes.
2. Servir como mecanismo de coordinación y control del flujo de ejecución.
3. Mantiene un resumen conversacional acumulativo, que condensa el contexto previo para optimizar el uso del LLM.

Cada agente lee exclusivamente la información necesaria para su análisis y escribe únicamente en los campos que le corresponden, sin dependencia directa de otros agentes. Esta interacción indirecta mediante el estado permite una orquestación desacoplada, trazable y extensible.

Las decisiones de enrutamiento se realizan a través de variables de control incluidas en el propio estado, que indican si un análisis requiere evaluación adicional y cuál debe ser el siguiente agente a ejecutar. De este modo, el flujo de ejecución emerge dinámicamente a partir de las salidas analíticas.

6.3.1. Estrategias de limpieza del estado

Para garantizar coherencia analítica y evitar contaminación entre escenarios, el sistema implementa dos estrategias diferenciadas de limpieza del estado:

- **Reset completo de caso (reset_full_case):** se activa cuando el sistema detecta el inicio de un nuevo escenario operativo (por ejemplo, un nuevo motor o aeronave). Este mecanismo elimina completamente el contexto previo, incluyendo historial analítico, resumen conversacional, salidas de agentes, datos de sensores y buffers, restableciendo el estado a una condición inicial controlada.
- **Reset por iteración (reset_state_iteration):** se ejecuta automáticamente en cada nueva consulta, limpiando únicamente la información transitoria asociada a la iteración actual, preservando el contexto relevante acumulado (historial por agente, resumen conversacional y datos de sensores).

6.4. Encadenamiento condicional entre agentes

El sistema permite **encadenamientos automáticos entre agentes** (*follow-up*) cuando se detectan condiciones específicas durante la ejecución:

- **PreRUL → RUL:** cuando el usuario solicita explícitamente el cálculo de RUL y existen datos.
- **RUL → Criticidad:** cuando la predicción de RUL es inferior a un umbral crítico ($RUL < 20$).
- **Tecnico → Criticidad:** si se identifica posible impacto operacional.
- **Regulación → Criticidad:** ante potencial incumplimiento normativo con impacto operativo.
- **Criticidad → Reparación:** si la severidad evaluada es *HIGH* o *CRITICAL*.

Si no se detectan riesgos relevantes, el flujo se dirige directamente al **FinalAgent**. Este encadenamiento condicional integra predicción, conocimiento documental y reglas de seguridad operacional en un único flujo coherente.

6.5. Composición final de resultados

El **FinalAgent** actúa como sintetizador del sistema. Su función es integrar exclusivamente las salidas producidas por los agentes especializados, sin generar información adicional. La respuesta final se compone siguiendo una jerarquía explícita de fuentes, priorizando los resultados según el tipo de consulta y el análisis realizado.

El resultado es una respuesta única, coherente y adaptada a la intención del usuario, que evita redundancias y contradicciones.

6.6. Ciclo de ejecución del sistema

Durante la fase de desarrollo y validación, AeroGPT se ejecutó inicialmente en un entorno local controlado mediante un módulo principal (main.py) que implementa un bucle interactivo. Este enfoque permitió depurar de forma incremental la lógica de orquestación, el comportamiento de los agentes y la correcta gestión del estado compartido.

El ciclo de ejecución del sistema, representado de forma esquemática en la Ilustración 4, sigue las siguientes etapas:

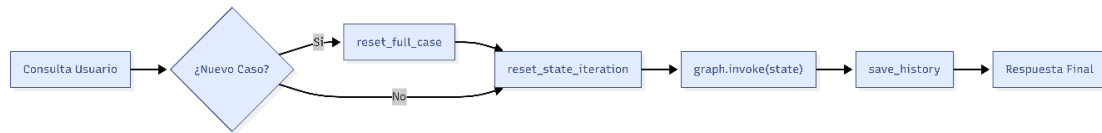


Ilustración 4: Ciclo de ejecución sistema completo

1. **Recepción de la consulta del usuario** en lenguaje natural.
2. **Inicialización o limpieza del estado**, aplicando, según el caso, un reset completo de escenario o una limpieza por iteración.
3. **Ejecución del grafo** (Ilustración 3):
 - 3.1. Clasificación inicial por el SupervisorAgent.
 - 3.2. Ejecución del agente especializado seleccionado
 - 3.3. Evaluación de condiciones de escalado.
4. **Persistencia del contexto y actualización del historial.**
5. **Síntesis final de resultados** mediante el FinalAgent.

Una vez validado el comportamiento en este entorno, se implementó la interfaz de producción con **Streamlit** que sirve como punto de acceso principal para usuarios finales. Esta interfaz web reutiliza íntegramente la lógica de orquestación del sistema, proporcionando una experiencia de usuario completa con chat interactivo, configuración en tiempo real y visualización de resultados, manteniendo la separación entre la capa de presentación y la lógica interna de los agentes.

6.7. Ventajas del enfoque multiagente

La arquitectura propuesta aporta múltiples beneficios:

- Alta especialización y precisión analítica.
- Composición adaptativa según la complejidad del caso.
- Separación clara de responsabilidades.
- Elevada trazabilidad y capacidad de depuración.
- Escalabilidad estructural para añadir nuevos agentes o fuentes sin rediseñar el sistema.

En conjunto, este capítulo muestra cómo la orquestación multiagente constituye el elemento integrador que convierte a AeroGPT en un **sistema avanzado de análisis y asistencia basada en datos**, cerrando el ciclo entre ingestión, modelado, razonamiento y generación de conocimiento accionable.

7. Resultados y experimentación

Este capítulo presenta los resultados obtenidos con el **sistema AeroGPT completo** ejecutado mediante un caso experimental representativo, diseñado para evaluar su comportamiento en un escenario de degradación severa. El objetivo principal no es volver a analizar el rendimiento numérico del modelo de predicción de RUL (ya evaluada en el Capítulo 4), sino validar la integración funcional del sistema multiagente como un proceso completo: desde la ingestión de datos hasta la generación de decisiones operacionales fundamentadas.

El experimento se centra en una consulta que conduce a un valor extremo de Vida Útil Restante (RUL = 0), lo que permite verificar el correcto funcionamiento de los mecanismos de escalado y coordinación entre agentes.

7.1. Definición del escenario experimental

La consulta planteada al sistema es la siguiente:

“Quiero calcular el RUL de un motor en condiciones FD004, con todas las configuraciones a 20000, y todos los sensores a 1000 menos el 1 que está a 4.”

Este escenario se selecciona por su carácter deliberadamente severo, ya que permite evaluar si el sistema es capaz de activar correctamente los mecanismos de escalado y de combinar información predictiva con conocimiento documental en condiciones límite:

- Se emplea el subconjunto **FD004**, el más complejo del dataset CMAPSS.
- Los valores de configuración y sensores se sitúan en rangos extremos, incompatibles con operación normal.
- La consulta fuerza la activación de un **flujo completo de análisis multiagente**.

La *Ilustración A.3* muestra el desarrollo completo de la conversación, incluyendo las decisiones internas del sistema y el historial de salidas por agente.

7.2. Flujo de ejecución y decisiones internas

7.2.1. Preprocesado de datos (PreRulAgent)

Ante la consulta inicial, el supervisor enruta a PreRulAgent, quien clasifica la intención del usuario como una acción de actualización (*Update*). A partir del texto proporcionado, interpreta los valores de sensores y configuraciones, inicializa un DataFrame de mediciones y registra una nueva fila correspondiente a un ciclo operativo.

El estado del sistema se actualiza con esta información y queda seleccionado el modelo correspondiente al escenario FD004. Cuando el usuario solicita explícitamente el cálculo de RUL (*Calculate*), el agente valida que existen datos suficientes y deriva el flujo hacia el agente de predicción.

Este comportamiento confirma que el sistema es capaz de interpretar correctamente instrucciones en lenguaje natural y transformarlas en estructuras de datos válidas para el análisis predictivo.

7.2.2. Predicción de RUL (RulAgent)

El **RulAgent** ejecuta su pipeline interno sobre los datos acumulados:

- Generación sintética de secuencia temporal hasta completar la ventana mínima.
- Normalización y preprocesado de las señales.
- Inferencia mediante el modelo GRU entrenado para FD004.
- Aplicación de calibración post-predicción.

El resultado obtenido es:

Predicted RUL: 0.0

Este resultado, aunque extremo, es coherente con los valores introducidos por el usuario, deliberadamente incompatibles con una operación normal. La obtención de un RUL nulo permite comprobar que el modelo y el pipeline analítico responden de forma consistente ante condiciones de fallo inminente.

7.3. Evaluación de criticidad y escalado automático

Al detectarse un RUL inferior al umbral crítico definido, el sistema activa el **CriticidadAgent**. Este agente integra la predicción numérica con evidencia documental y reglas operacionales, produciendo la siguiente clasificación:

- **Severidad: CRITICAL**

- **Despacho permitido: False**

La clasificación como CRITICAL y la decisión de no permitir el despacho muestran que el sistema no se limita a producir predicciones numéricas, sino que es capaz de traducirlas en decisiones operacionales alineadas con criterios de seguridad aeronáutica.

7.4. Recomendaciones de reparación

Ante una severidad clasificada como crítica, el flujo continúa hacia el **ReparacionAgent**. Este agente genera un plan de actuación estructurado que incluye:

- Retirada inmediata del motor para inspección.
- Revisión exhaustiva de los sistemas afectados.
- Aplicación de procedimientos de mantenimiento específicos con referencias a manuales técnicos y documentación normativa.

Este resultado demuestra la capacidad del sistema para integrar información procedente de múltiples fuentes (predicción RUL, documentación técnica y reportes históricos) y transformarla en recomendaciones accionables, algo esencial en contextos reales de mantenimiento.

7.5. Síntesis final y visualización

El **FinalAgent** integra las salidas previas en una **respuesta única y coherente** para el usuario, confirmando el RUL estimado, describiendo los indicadores de degradación y proponiendo acciones inmediatas.

Durante la experimentación se habilitó una visualización que permite mostrar u ocultar:

- Las **decisiones internas** del sistema.
- El **historial por agente** con las salidas JSON estructuradas individualmente.

De esta forma, se verifica que AeroGPT puede actuar como un asistente integral de soporte a la decisión, proporcionando no solo información, sino también contexto, justificación y trazabilidad.

7.6. Discusión

El experimento realizado permite validar que la arquitectura propuesta funciona como un sistema integrado y coherente. A partir de una consulta en lenguaje natural, AeroGPT ha sido capaz de interpretar datos de entrada, ejecutar un modelo predictivo, evaluar su resultado desde una perspectiva operacional y generar recomendaciones técnicas fundamentadas.

El flujo observado confirma la correcta coordinación entre agentes y la utilidad del estado compartido como mecanismo de integración. Asimismo, la capacidad de escalar automáticamente desde la predicción numérica hacia el análisis de criticidad y reparación evidencia la flexibilidad del enfoque multiagente.

Aunque se trata de un escenario sintético y extremo, los resultados obtenidos demuestran que la metodología es aplicable a contextos reales donde la toma de decisiones debe basarse en múltiples fuentes de información y criterios de seguridad.

8. Conclusiones generales y líneas futuras

Este Trabajo Fin de Máster ha puesto de manifiesto el potencial de la integración de técnicas de **Big Data e Inteligencia Artificial** para abordar problemas industriales complejos caracterizados por la coexistencia de grandes volúmenes de datos heterogéneos. A lo largo del proyecto se ha demostrado que la explotación

conjunta de series temporales multivariantes de sensores y documentación técnica y normativa permite generar conocimiento de mayor valor que el análisis aislado de cada fuente.

Desde la perspectiva del **Big Data**, uno de los aspectos centrales ha sido el diseño de pipelines robustos de ingestión, limpieza y transformación de datos. En el ámbito numérico, se han aplicado procesos sistemáticos de control de calidad, normalización y construcción de secuencias temporales sobre el dataset CMAPSS, garantizando la coherencia entre conjuntos de entrenamiento y test y evitando cualquier forma de data leakage. De manera análoga, para los datos textuales se han desarrollado pipelines diferenciados capaces de procesar documentos PDF complejos y bases de datos operacionales, transformándolos en representaciones vectoriales eficientes y trazables. Esta fase de preparación constituye un elemento fundamental para la fiabilidad del sistema final.

En relación con las técnicas de Inteligencia Artificial, el trabajo ha integrado modelos de **Deep Learning** para la predicción de la Vida Útil Restante (RUL) con modelos de lenguaje de gran escala apoyados en Retrieval-Augmented Generation (RAG). La evaluación comparativa de arquitecturas recurrentes, junto con esquemas de validación cruzada orientados a estabilidad, ha permitido seleccionar modelos GRU que ofrecen un equilibrio óptimo entre precisión y robustez. La incorporación de una calibración post-predicción ha reforzado la utilidad práctica del modelo, corrigiendo sesgos sistemáticos sin comprometer la independencia de los datos de test.

La principal aportación del proyecto reside en el diseño e implementación de una **arquitectura multiagente** orientada a datos, capaz de coordinar de forma dinámica análisis predictivos, recuperación documental y reglas operacionales. AeroGPT no se limita a generar texto, sino que integra procesos analíticos y decisiones fundamentadas en evidencia, proporcionando respuestas trazables y contextualizadas. Este enfoque permite abordar consultas técnicas complejas mediante un flujo adaptativo que combina distintos tipos de conocimiento y técnicas analíticas.

Desde el punto de vista práctico, el sistema desarrollado resulta **aplicable a escenarios reales de mantenimiento predictivo y soporte a la decisión**. Las salidas generadas no solo ofrecen estimaciones numéricas de RUL, sino que las complementan con análisis técnicos, regulatorios y operativos, facilitando su interpretación por parte de ingenieros y responsables de mantenimiento. Además, la modularidad del diseño permite extender fácilmente el sistema a otros dominios industriales con requisitos similares de seguridad y trazabilidad.

No obstante, el trabajo presenta también ciertas limitaciones. El modelo de predicción se ha evaluado sobre un dataset sintético, por lo que su rendimiento podría variar al aplicarse sobre datos reales con mayor ruido y variabilidad. Asimismo, la dependencia de modelos de lenguaje externos implica costes computacionales y potenciales riesgos de alucinación que deben gestionarse mediante controles adicionales.

Como líneas futuras se proponen:

- Incorporar datos reales de mantenimiento para validar el modelo en entornos operativos.
- Integrar técnicas de explicabilidad para mejorar la interpretación de las predicciones de RUL.
- Ampliar el repositorio documental con nuevas fuentes y estándares aeronáuticos.
- Incluir métricas de confianza y calibración dinámica en la salida de los agentes.

En conclusión, este trabajo demuestra que la combinación rigurosa de ingeniería de datos, modelos avanzados de IA y arquitecturas multiagente constituye una aproximación eficaz para sistemas de soporte a la decisión en entornos críticos.

AeroGPT materializa los conceptos aprendidos durante el máster en una solución funcional, extensible y alineada con necesidades reales, estableciendo una base sólida para futuras aplicaciones industriales.

9. Bibliografía

NASA – National Aeronautics and Space Administration. (s. f.). *C-MAPSS Turbofan Engine Degradation Simulation Data Sets (FD001–FD004)*. Intelligent Systems Division – Prognostics Center of Excellence. <https://www.nasa.gov/intelligent-systems-division/discovery-and-systems-health/pcoe/pcoe-data-set-repository/>

NASA Aviation Safety Reporting System (ASRS). (s. f.). *ASRS Online Database*. <https://asrs.arc.nasa.gov/search/database.html>

Federal Aviation Administration (FAA). (s. f.). *Service Difficulty Reporting System (SDRS)*. <https://sdrs-preprod.faa.gov/>

Federal Aviation Administration (FAA). (s. f.). *Advisory Circulars (ACs)*. https://www.faa.gov/regulations_policies/advisory_circulars/index.cfm/go/document.list/topicID/140

European Union Aviation Safety Agency (EASA). (s. f.). *EASA Regulations and Certification Specifications*. <https://www.easa.europa.eu/en/regulations>

Airbus. (s. f.). *FAST Magazine – Technical and operational publications*. <https://www.aircraft.airbus.com/en/fast-magazine-articles>

Scikit-learn Developers. (s. f.). *Scikit-learn Documentation*. <https://scikit-learn.org/stable/>

TensorFlow/Keras. (s. f.). *Keras Documentation*. <https://keras.io/>

OpenAI. (2024). *Text Embeddings API Documentation*. <https://platform.openai.com/docs/guides/embeddings>

LangChain. (2024). *LangChain Documentation*. <https://python.langchain.com/>

LangGraph. (2024). *LangGraph Documentation*. <https://langchain-ai.github.io/langgraph/>

PyTorch. (s. f.). *PyTorch Documentation*. <https://pytorch.org/docs/>

Meta AI. (s. f.). *FAISS – Facebook AI Similarity Search Documentation*. <https://faiss.ai/>

10. Anexos

El código fuente del proyecto AeroGPT está disponible en el siguiente repositorio de GitHub:

<https://github.com/davidcp199/Aerogpt>

Fuente	Formato	Documentos	Vectorstore	Pipeline
FAA Advisory Circulars	PDF	57 archivos	regulatory_store	A
EASA (CS, Part, AMC/GM)	PDF	4 archivos	regulatory_store	A
Airbus FAST Magazine	PDF	43 archivos	technical_store	A
NASA ASRS	Excel	Variable	asrs_store	B
FAA SDR	Excel	Variable	sdr_store	B

Tabla A.1: Resumen de fuentes y vectorstores usados en el sistema. [volver a 3.1]

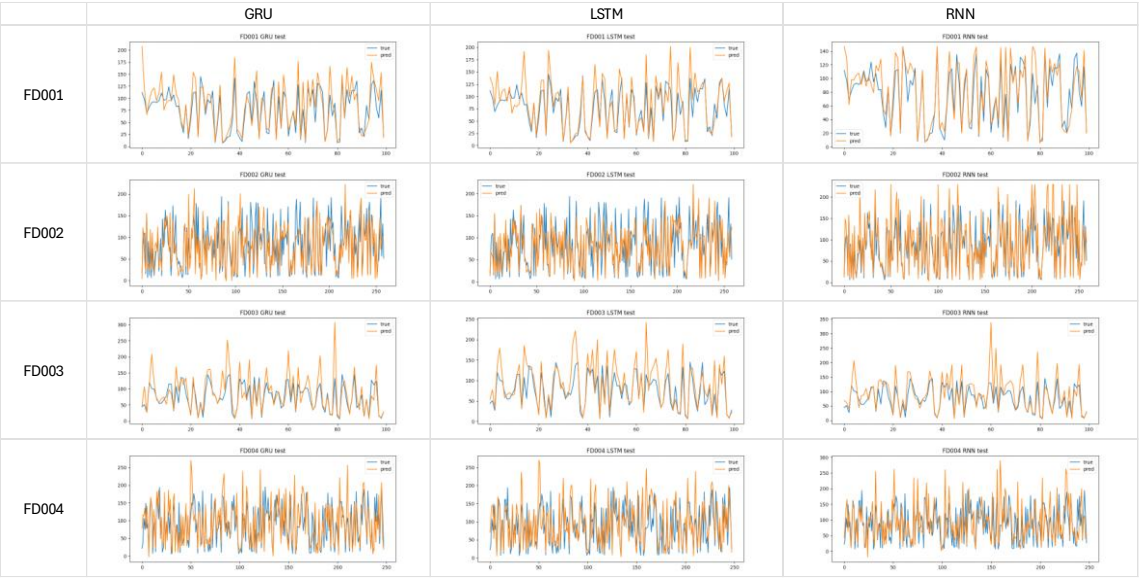


Ilustración A.1: Gráfica valores Reales vs Predichos de RUL por Arquitectura y FD. [volver a 4.4]

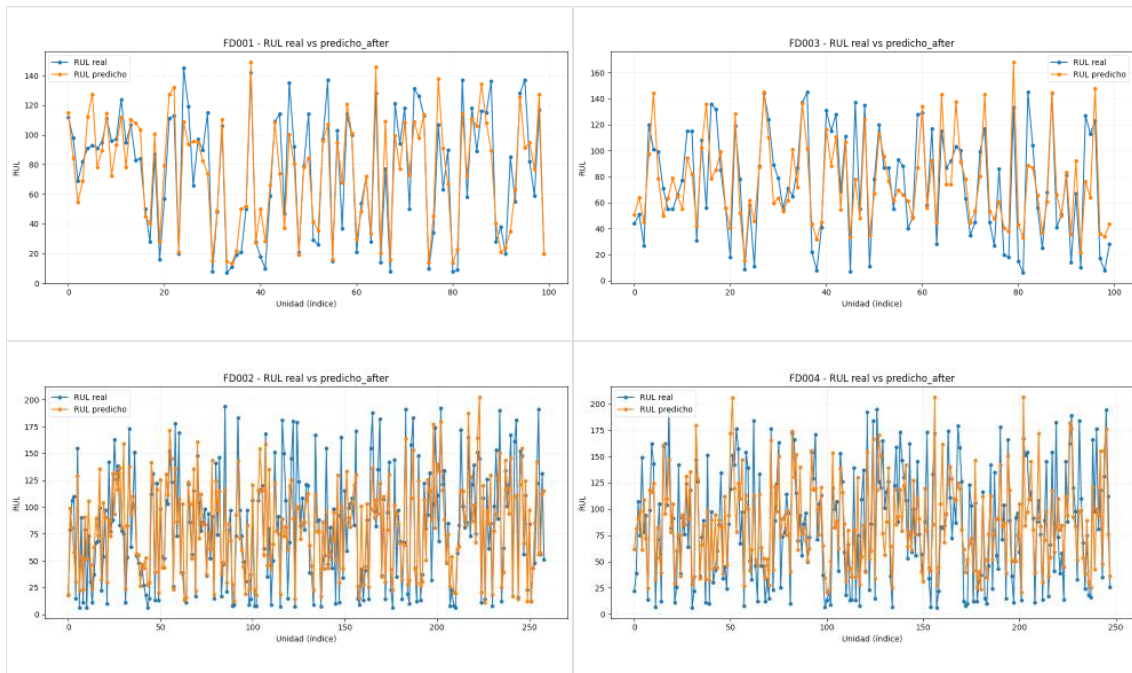


Ilustración A.2: Gráficas RUL real vs predicho modelo refinado GRU para FD001, FD002, FD003 y FD004. [\[volver a 4.6\]](#)

Métrica	Sin Calibrar	Calibrado
MAE	27.983	24.837
RMSE	39.546	32.455
NASA score	125,399,595	290,771

Tabla A.2: Comparación métricas GRU antes y después de calibración. [\[volver a 4.6\]](#)

```
{
  "applicability": ["CS-25"],
  "aircraft_applicability": true,
  "dispatch_relevance": false,
  "regulations": [
    {
      "authority": "EASA",
      "reference": "CS 25.1309",
      "topic": "System safety",
      "constraint": "Evaluación de integridad ..."
    }
  ],
  "operational_limitations": ["No operar con sistema X degradado"],
  "compliance_risk": "LOW",
  "summary": "Según CS 25.1309 y guías asociadas, ..."
}
```

Tabla A.3: Ejemplo JSON salida RegulationAgent. [\[volver a 5.2\]](#)

```

{
  "affected_system": "Electrical generation",
  "flight_phase": "Cruise",
  "severity": "HIGH",
  "operational_risk": "Pérdida parcial de generación con riesgo de..."
  "references": [
    "ASRS report 12345: pérdida intermitente de generador",
    "SDR 67890: fallo de excitador"
  ],
  "recommendations": [
    "Proceder a verificación en tierra del sistema de excitación; registrar P/N y horas.",
    ...
  ]
}

```

Tabla A.4: Ejemplo JSON salida CriticidadAgent. [volver a 5.2]

```

{
  "system_affected": "Hydraulic system (ATA 29)",
  "flight_phase": "Cruise",
  "severity": "HIGH",
  "recommended_actions": [
    "Aislar la línea hidráulica afectada; aplicar procedimiento de bypass según AMM (si aplica).",
    "Reemplazar acumulador hidráulico y verificar integridad de acoplamientos; registrar P/N y horas.",
    "Realizar prueba funcional en tierra siguiendo checklist TSM; verificar presiones y fugas."
  ],
  "references": [
    "AMM 29-01-10", "TSM 29-05-23"
  ],
  "notes": "Supuestos: aeronave narrow-body genérica; no se dispone de intervención previa. Requiere verificación con AMM del fabricante."
}

```

Tabla A.5: Ejemplo JSON salida ReparacionAgent. [volver a 5.2]



Ilustración A.3: Captura prueba completa del sistema con interfaz Streamlit. [volver a 7.1]

Categoría	Tecnología	Propósito
Framework de Orquestación de Agentes	LangChain v1.1.3	Framework principal para construir agentes conversacionales.
	LangGraph	Especializado en construir grafos de agentes con flujos condicionales.
Modelos de Lenguaje (LLMs)	OpenAI GPT-4 API v2.8.1	Modelos de lenguaje para tareas estructuradas y creativas.
Sistema RAG	FAISS v1.13.1	Motor de búsqueda vectorial para recuperación de información.
	OpenAI Embeddings	Generación de representaciones vectoriales de texto.
Deep Learning y Predicción de RUL	PyTorch v2.5.1	Framework de deep learning para modelos de predicción de vida útil.
Machine Learning Tradicional	Scikit-learn v1.6.1	Librería para preprocesamiento y calibración de modelos.
Procesamiento de Datos	Pandas v2.2.3	Análisis y manipulación de datos tabulares.
	NumPy v2.1.2	Operaciones numéricas y arrays multidimensionales.
	SciPy v1.15.3	Funciones científicas y estadísticas complementarias.
Procesamiento de Documentos	pdfplumber	Extracción de texto de documentos PDF.
	openpyxl	Lectura de archivos Excel con datos estructurados.
	Chunking de documentos	Procesamiento personalizado que divide textos en fragmentos con solapamiento.
Interfaz de Usuario	Streamlit v1.52.2	Framework para crear la aplicación web interactiva.
Validación y Estructuración de Datos	Pydantic	Validación y serialización de datos estructurados.
Visualización y Análisis	Matplotlib v3.9.2	Generación de gráficos para evaluación de modelos.
	Seaborn v0.13.2	Visualizaciones estadísticas mejoradas.
Configuración y Utilidades	PyYAML v6.0.2	Gestión de archivos de configuración.
	python-dotenv v1.2.1	Gestión de variables de entorno sensibles.
	tiktoken v0.12.0	Tokenización para modelos OpenAI.
Registro de Modelos y Herramientas	LangSmith v0.4.48	Observabilidad y debugging de cadenas LangChain.
	Logging	Sistema de logging nativo de Python.

Tabla A.6. Tecnologías y versión, junto con su uso en el sistema